GALILEO Graph Databases Group

Implementation of GALOIS Features – Second Check

(Queries 18–30, Replication of Table 3 – Third Column)

Francesco Pivotto 2158296
Giorgia Amato 2159999
Alessio Demo 2142885

November 23, 2025

Contents

1 Introduction

Large Language Models (LLMs) increasingly support structured data extraction using declarative queries such as Natural Language (NL) and SQL. However, when queries become more complex and require structured tabular outputs with filtering, joins, and aggregations, direct prompting of LLMs leads to inaccurate or incomplete results. As shown in the Galois paper, both NL and SQL prompting struggle with factual correctness, tuple completeness, and recall accuracy.

To address these limitations, the Galois system introduces a novel query execution framework that uses the LLM as a storage layer, while separating the execution pipeline into logical and physical operators, similar to traditional DBMSs. Instead of issuing a single LLM request, Galois decomposes SQL queries into a sequence of structured scan and filtering operations that improve recall, tuple completeness, and overall output consistency.

Scope of This Deliverable. In this checkpoint, we implement **GaloisWO** (Galois Without Optimizations), which is the baseline version of the Galois framework introduced in Section 5 of the paper. GaloisWO does not apply optimization strategies such as confidence-based condition pushdown, operator selection, cost-quality balancing, or metadata-driven planning. Instead, it follows a straightforward execution strategy:

- Retrieve all possible key values for a table using iterative prompting.
- For each key, individually retrieve remaining attribute values (cell-by-cell tuple completion).
- No condition pushdown or selection filtering during scan phase.

This deliverable covers:

- Implementation of the GaloisWO execution strategy for queries 18–30.
- Replication attempt of the **third column of Table 3** from the paper.
- Informal demonstration of execution behavior and quality scores.

2 Implementation of GALOISWO Baseline

This section describes the implementation of **GALOISWO** (Galois Without Optimizations), the baseline version of the Galois framework implemented for this checkpoint. Unlike the full Galois system, GaloisWO does not apply any logical or physical optimizations such as confidence-based pushdown, selective operator choice, metadata reasoning, or cost–quality trade-off. Instead, it adopts a simple two-stage execution pipeline:

1. **Key Extraction**: retrieve all possible primary keys of the target table using iterative prompting.
2. **Tuple Completion**: for each retrieved key, prompt the LLM to obtain remaining attribute values (one tuple at a time).

2.1 Key Retrieval Phase (First Stage)

2.2 Table-Scan Execution Phase (Second Stage)

In the Galois paper, two physical scan operators are introduced: *Table-Scan* and *Key-Scan*. While the full Galois system dynamically selects between them according to confidence and metadata, GaloisWO uses only **Table-Scan** in a fully deterministic manner, with no optimization or pushdown logic.

This behavior is clearly reflected in our implementation, specifically in the `table_scan()` method of the `GaloisExecutor` class. The execution shows the following key characteristics:

- **No condition pushdown:** The executor uses the original SQL query, but does not apply any optimization selection. The WHERE clause is either used as-is or fully omitted if not available. There is no selection of pushable conditions based on metadata or confidence values.
- **Schema-aware attribute discovery:** The executor retrieves full table attribute lists using the `GaloisWOSchemaManager`, which is a thin wrapper around the standard schema manager but isolates the GaloisWO namespace.
- **First Prompt and Iterative Prompt Generation:** The system constructs the first and iterative human prompts using: `build_table_scan_first_prompt()` and `build_table_scan_iter_prompt()` from `galois_prompts.py`.

  These prompts enforce:

  - valid JSON-only output,
  - table-aware attribute listing,

  - a maximum of 20 tuples per iteration,
  - and no natural language explanations.

- **Iterative Expansion:** The executor issues multiple LLM calls (up to `max_iter`, default 4), collects new tuples, and stops when no new data is returned.
- **Duplicate Removal and JSON Recovery:** The system hashes tuple contents (via `normalise_row()`) to track and deduplicate results across iterations. It also attempts to recover valid JSON even from partially truncated or malformed LLM output.

Unlike Key-Scan, which explicitly extracts primary keys first and then populates attributes one by one, Table-Scan retrieves complete tuples directly using LLM in multiple iterations.

2.3 Table vs Key-Scan in GaloisWO

3 Contributions

Alessio Demo (Badge number: 2142885)

Francesco Pivotto (Badge number: 2158296)

Giorgia Amato (Badge number: 2159999)